Informe de Laboratorio de Minería de Datos: Análisis Exploratorio del Healthcare Dataset

Sesión 5 · Evaluación (instrumento del PEA)

Manuel José Rivera Guzmán y Julián Andrés Correa Cuevas
Corporación Universitaria Minuto de Dios
Programa: Ingeniería de Sistemas / Minería de Datos
Curso: Minería de Datos
Docente: Estaban Ernesto Morales Castro
Octubre de 2026

Resumen

Se presenta un análisis exploratorio del Healthcare Dataset (Kaggle), compuesto por 54.966 admisiones hospitalarias después del proceso de limpieza (55.500 registros crudos). El estudio evalúa la calidad de los datos, caracteriza las variables demográficas, clínicas y administrativas mediante análisis univariado, y examina las relaciones entre variables y el monto de facturación (Billing Amount) mediante análisis bivariado y multivariado, incluyendo el enriquecimiento del dataset con un catálogo de medicamentos (Drugs_product). Los resultados muestran que las variables numéricas (edad, duración de la estancia) no se correlacionan linealmente con la facturación ($r \approx 0$), que no existen diferencias significativas en el monto facturado por tipo de admisión, género ni resultado de prueba, y que la única asociación significativa se observa para la condición médica Cáncer (menor facturación, $p = 0{,}004$). El cruce con el catálogo Drugs muestra que cada paciente recibe exactamente un medicamento, que los medicamentos son mutuamente excluyentes ($r \approx -0{,}25$) y que la facturación se asocia levemente con las facturas negativas ($r = -0{,}08$). Se identifican 106 facturas con monto negativo y se proponen segmentos de facturación y de edad como nuevas variables.

Palabras clave: minería de datos, análisis exploratorio, healthcare dataset, facturación hospitalaria, calidad de datos, cruce de datos, catálogo de medicamentos.

1. Contexto

1.1 Pregunta de negocio

¿Qué factores se asocian con el monto de facturación de las admisiones hospitalarias, qué patrones caracterizan a los pacientes y qué aporta el enriquecimiento con el catálogo de medicamentos?

1.2 Objetivos del análisis

- Caracterizar la distribución de las variables demográficas, clínicas y administrativas del dataset.
- Evaluar la calidad de los datos heredada de las sesiones 3 y 4 (nulos, duplicados, outliers).
- Identificar asociaciones entre las variables y el monto de facturación (Billing Amount).
- Enriquecer el dataset con el catálogo Drugs_product y analizar el cruce.
- Proponer nuevas variables y segmentos, y validar al menos dos hipótesis explícitas con datos.

1.3 Dataset y proceso

Fuente: Healthcare Dataset (prasad22, 2024) de Kaggle, con 55.500 admisiones y 15 columnas originales. Tras la limpieza quedan 54.966 registros y 16 columnas (se añade Factura_Negativa). El enriquecimiento con el catálogo Drugs_product (FDA) genera un dataset de 54.966 filas y 63 columnas, incorporando one-hot de variables categóricas, niveles (Medical Condition, Admission Type), segmento y edad robusta, y atributos farmacológicos por medicamento (PHARM_CLASSES, DOSAGEFORMNAME, ROUTENAME, DEASCHEDULE).

2. Calidad de Datos

La calidad de los datos se hereda de las sesiones 3 y 4. A continuación se resumen los hallazgos y las decisiones tomadas.

Tabla 1

Resumen de calidad de datos del dataset

Indicador	Crudo	Limpio	Decisión
Filas	55.500	54.966	Eliminar 534 duplicados
Duplicados	534	0	drop_duplicates( )
Nulos	0	0	Sin imputación requerida
Facturas negativas	108	106 marcadas	Marcar, no imputar (dato sensible)
Edades fuera de rango	0	0	Sin acción
Outliers (IQR)	0	0	Sin eliminación

Nulos: no se encontraron valores faltantes en ninguna de las 16 variables del dataset limpio, por lo que no fue necesaria imputación.

Duplicados: se detectaron 534 filas duplicadas en el crudo y se eliminaron antes de las cuentas posteriores.

Outliers (IQR): aplicando la regla 1,5·IQR sobre Age y Billing Amount (excluyendo facturas negativas) no se detectaron valores fuera de rango.

Facturas negativas: 106 registros presentan monto de facturación negativo. Se marcan en la variable Factura_Negativa sin modificar el monto original.

Dataset enriquecido (cruce): presenta 1.121.314 valores nulos, concentrados en las columnas farmacológicas (MNAR: información no disponible en el catálogo, no error de captura). No se imputan. Num_Medicamentos = 1 para todos los registros (cada paciente recibe exactamente un medicamento).

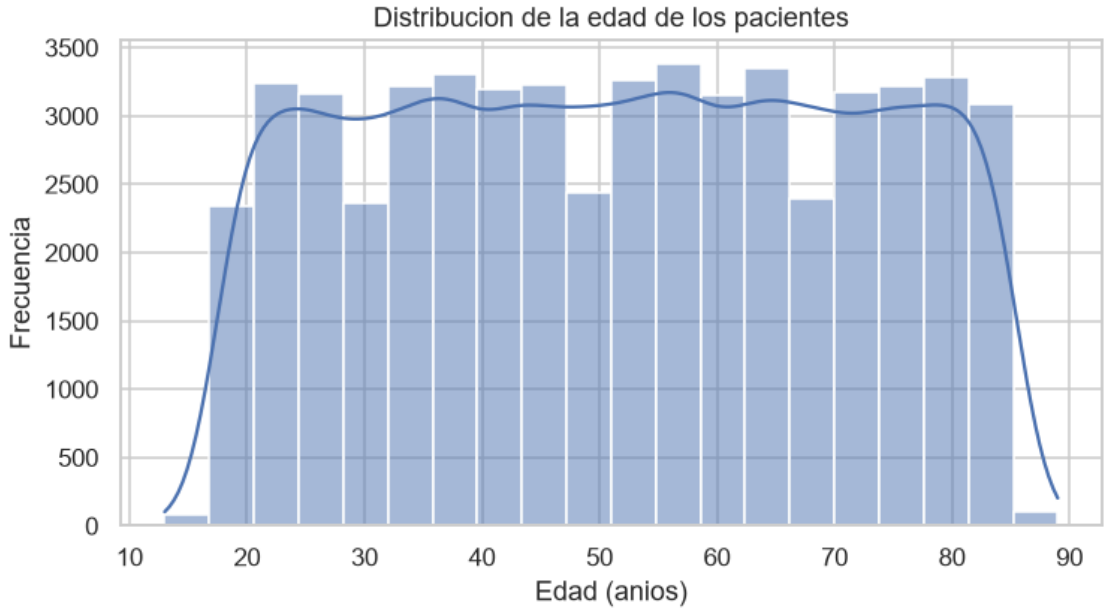
3. Análisis Univariado

Se presentan las estadísticas descriptivas de las variables numéricas principales y las figuras interpretadas.

Tabla 2
Estadísticas descriptivas de variables numéricas

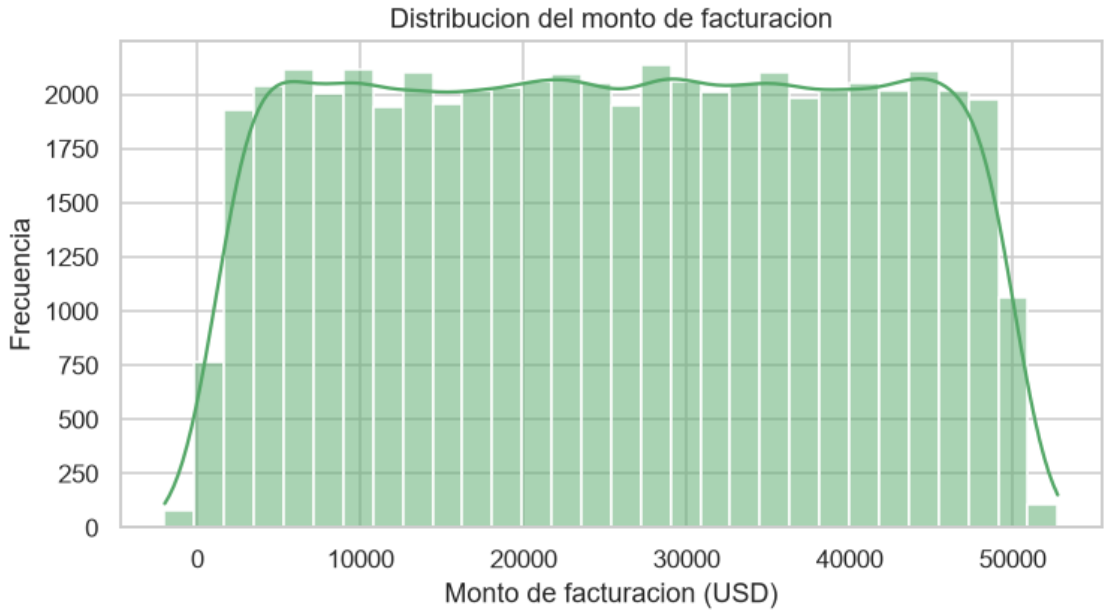
Variable	n	Media	Desv. estándar	Mín.	Máx.
Edad (años)	54.966	51,54	19,61	13	89
Monto de facturación (USD)	54.966	25.544,31	14.208,41	-2.008,49	52.764,28
Duración de estancia (días)	54.966	15,50	8,66	1	30

Figura 1
Distribución de la edad de los pacientes.



Nota. La edad se distribuye de forma aproximadamente uniforme entre los 13 y 89 años, sin concentración en un grupo etario específico.

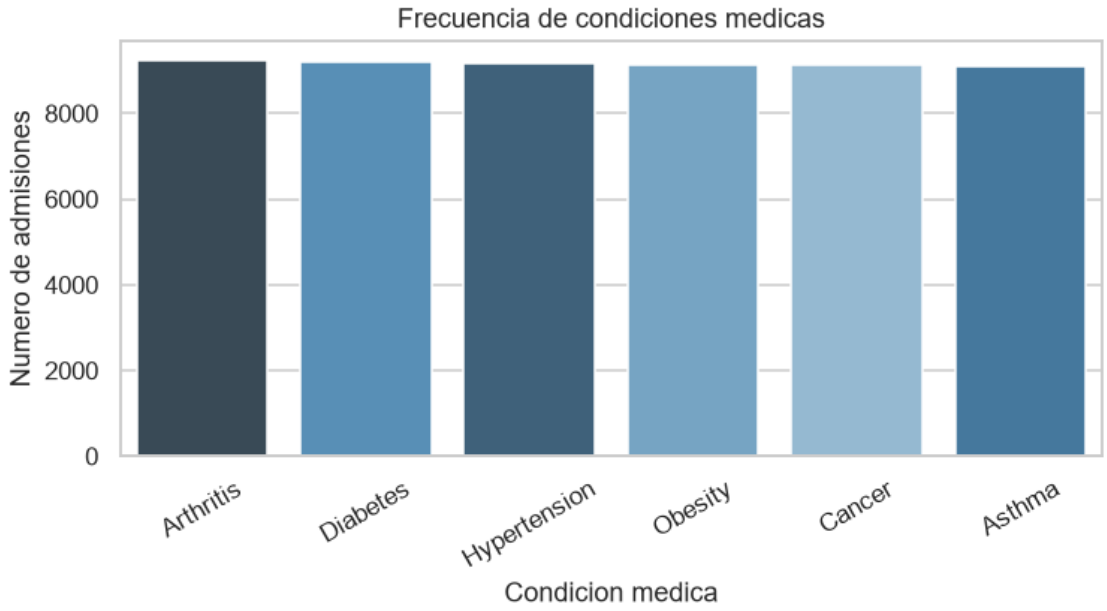
Figura 2
Distribución del monto de facturación.



Nota. El monto de facturación presenta una distribución aproximadamente uniforme entre 0 y 52.764 USD, con media de 25.544 USD. No se observa sesgo marcado ni outliers IQR.

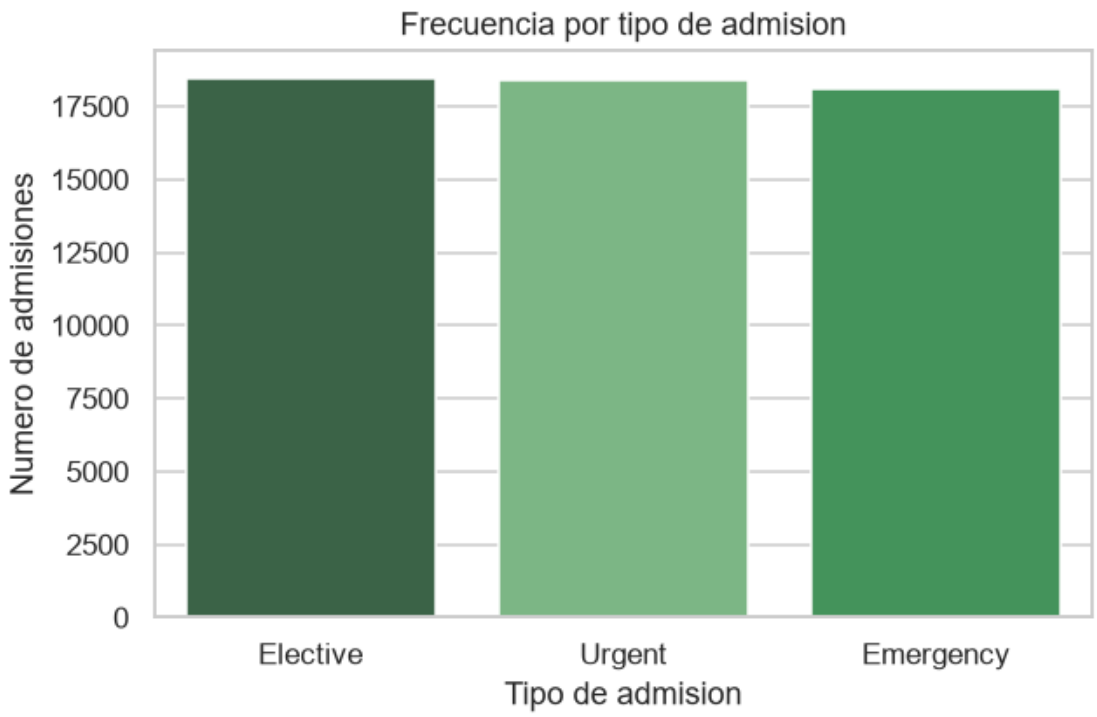
Las condiciones médicas se distribuyen de forma equilibrada: Arthritis (9.218), Diabetes (9.216), Hypertension (9.151), Obesity (9.146), Cancer (9.140) y Asthma (9.095). Los tipos de admisión son Elective (18.473), Urgent (18.391) y Emergency (18.102). Los resultados de prueba se reparten entre Abnormal (18.437), Normal (18.331) e Inconclusive (18.198). El género es prácticamente paritario (Male 27.496, Female 27.470).

Figura 3
Frecuencia de condiciones médicas.



Nota. Las seis condiciones médicas tienen frecuencias similares (~9.000 casos cada una), reflejando un diseño balanceado del dataset.

Figura 4
Frecuencia por tipo de admisión.

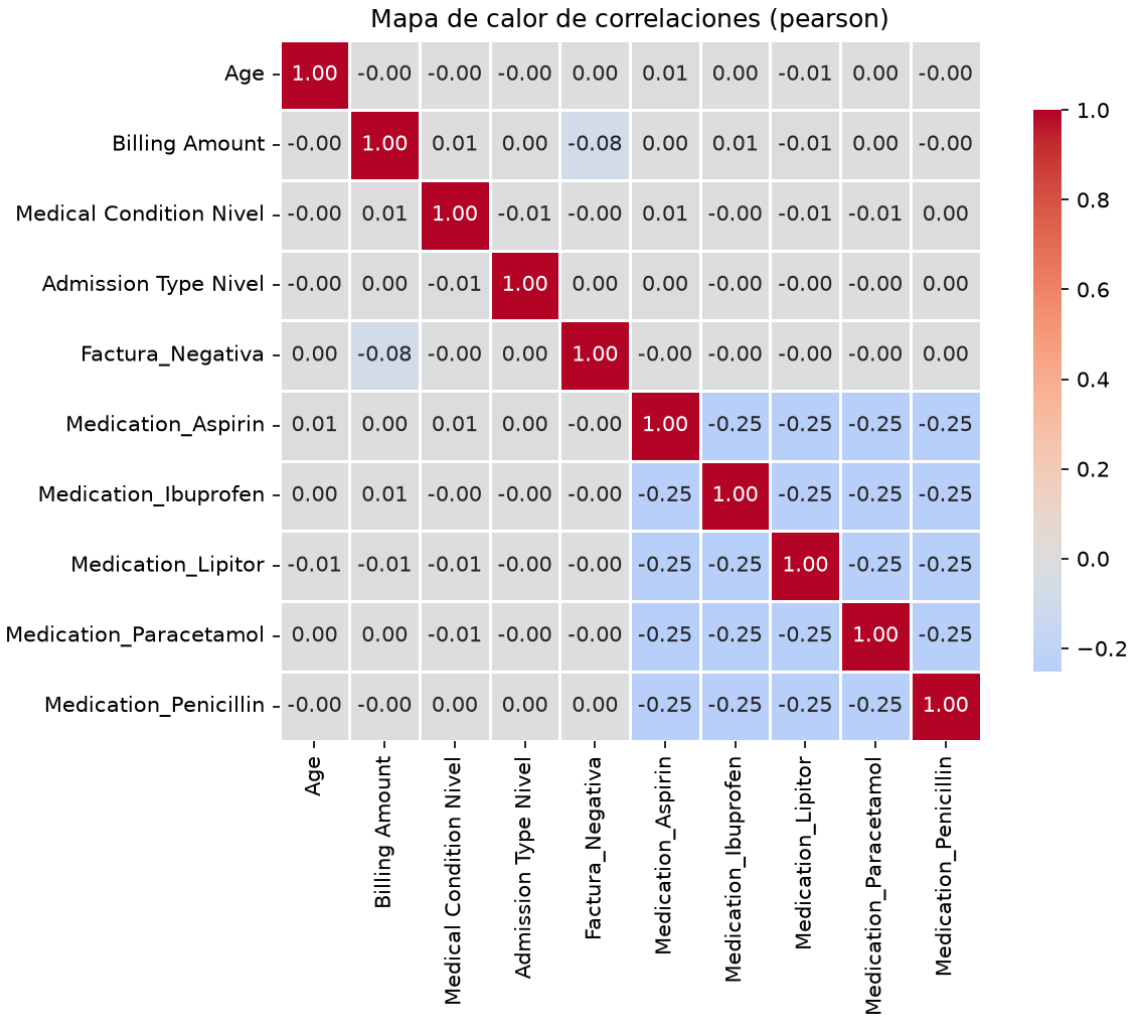


Nota. Los tres tipos de admisión (Elective, Urgent, Emergency) se presentan con frecuencias casi idénticas.

4. Análisis Bivariado y Multivariado

Se examinan las correlaciones entre variables numéricas y los cruces de la variable objetivo (Billing Amount) con variables categóricas, incluyendo el dataset enriquecido.

Figura 5
Mapa de calor de correlaciones (datos cruzados).



Nota. Heatmap sobre el dataset enriquecido (Age, Billing Amount, Medical Condition Nivel, Admission Type Nivel, Factura_Negativa y variables Medication_*). Las correlaciones más altas son estructurales: entre medicamentos son negativas (r ≈ -0,25) por ser mutuamente excluyentes (cada paciente recibe un solo medicamento). La única correlación con Billing Amount es con Factura_Negativa (r = -0,08). El resto de asociaciones son prácticamente nulas (|r| < 0,01).

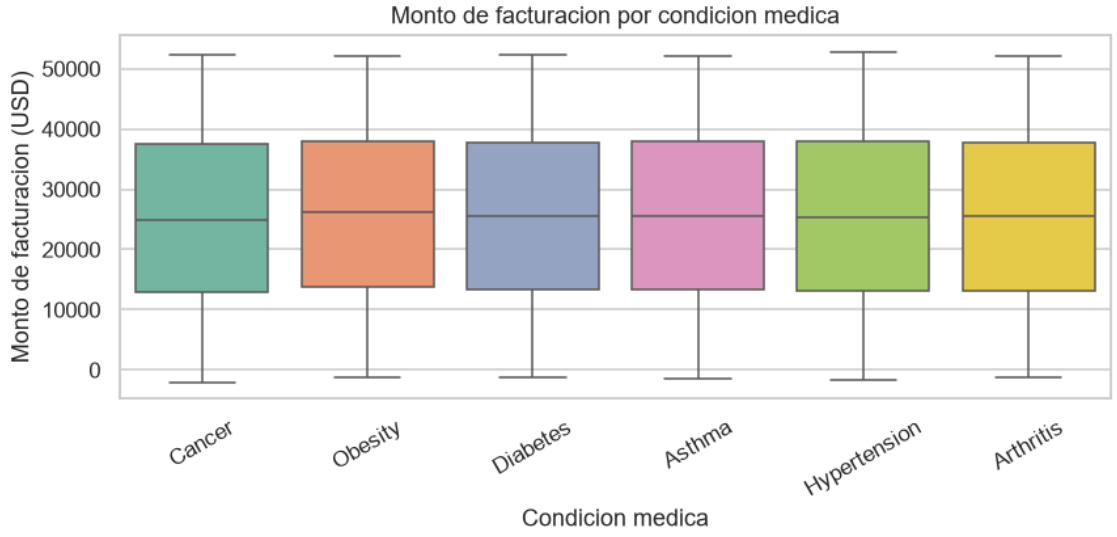
Tabla 3
Monto de facturación por condición médica

Condición médica	n	Media (USD)	Mediana (USD)
Arthritis	9.218	25.511,78	25.599,19
Asthma	9.095	25.633,46	25.628,41

Cancer	9.140	25.152,32	24.910,71
Diabetes	9.216	25.660,48	25.647,71
Hypertension	9.151	25.503,06	25.281,63
Obesity	9.146	25.804,36	26.145,58

Nota. Elaboración propia. La condición Cáncer presenta la media más baja.

Figura 6
Monto de facturación por condición médica.



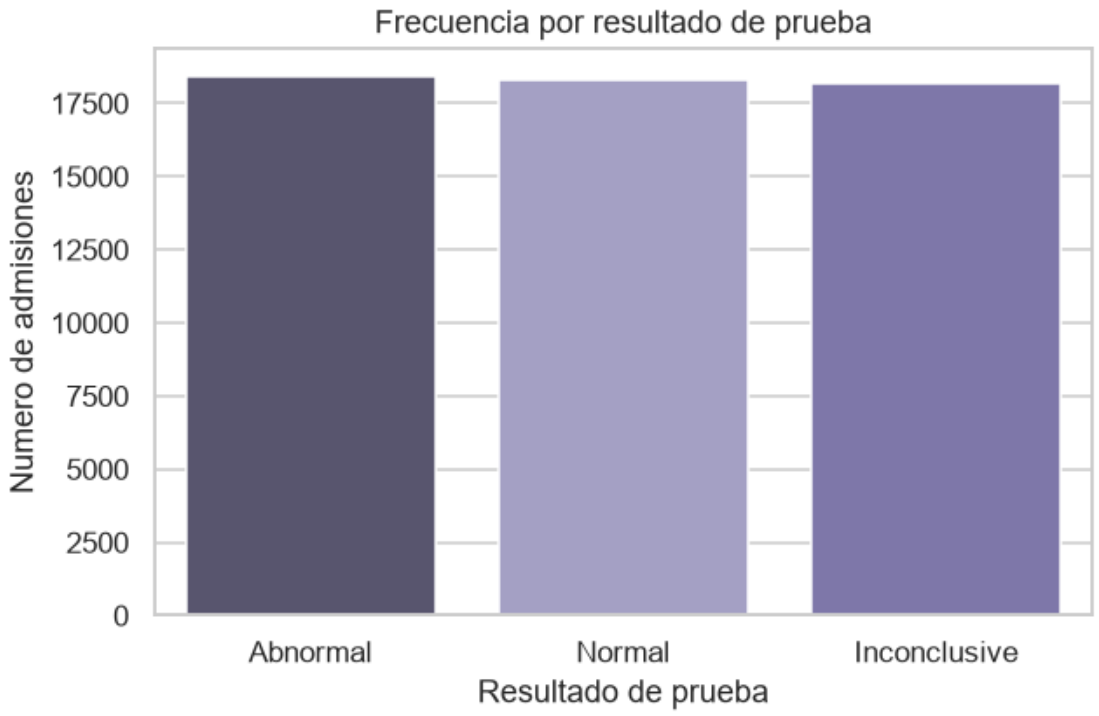
Nota. Los boxplots muestran distribuciones muy similares entre condiciones, con medianas cercanas a 25.500 USD. Cáncer presenta una mediana ligeramente inferior.

Tabla 4
Monto de facturación por tipo de admisión, género y resultado de prueba

Variable / Categoría	**n**	**Media (USD)**	**Mediana (USD)**
Admission Type - Elective	18.473	25.612,14	25.674,74
Admission Type - Emergency	18.102	25.505,33	25.413,78
Admission Type	18.391	25.514,53	25.543,56

- Urgent			
Gender - Female	27.470	25.474,10	25.427,85
Gender - Male	27.496	25.614,45	25.686,88
Test Results - Abnormal	18.437	25.544,04	25.602,59
Test Results - Inconclusive	18.198	25.637,34	25.699,28
Test Results - Normal	18.331	25.452,22	25.349,75

Figura 7
Frecuencia por resultado de prueba.



Nota. Los resultados de prueba (Abnormal, Normal, Inconclusive) se distribuyen de forma casi idéntica.

4.3 Datos cruzados: enriquecimiento con el catálogo Drugs

El dataset limpio se enriquece con el catálogo Drugs_product (FDA), que contiene información de medicamentos (clases terapéuticas, forma farmacéutica, vía de administración, clasificación DEA). El matching se realiza por nombre del medicamento (Aspirin, Ibuprofen, Lipitor, Paracetamol, Penicillin) y genera variables one-hot por medicamento más atributos farmacológicos.

Tabla 5

Cobertura del cruce healthcare–Drugs

Medicamento	Productos en catálogo	DEA Schedule	Vía	Sustancia activa
Aspirin	10.984	CIII	Oral	Acetylsalicylic acid
Ibuprofen	11.023	CII	Oral	Ibuprofen
Lipitor	11.038	No clasificado	Oral	Atorvastatin
Paracetamol	10.965	CII	Oral	Acetaminophen
Penicillin	10.956	No clasificado	Oral/Parenteral	Penicillin

Nota. Elaboración propia a partir de Drugs_product (FDA). Cada paciente recibe exactamente un medicamento (Num_Medicamentos = 1).

Los valores nulos en las columnas farmacológicas (1.121.314 en total) son de tipo MNAR (Missing Not At Random): la información no está disponible en el catálogo, no es un error de captura. No se imputan.

Las variables Medication_<medicamento> son mutuamente excluyentes: cada paciente tiene exactamente un medicamento, por lo que sus correlaciones son negativas ($r \approx -0,25$) y estructurales, no indicativas de asociación causal.

La clasificación DEA muestra que Ibuprofen y Paracetamol son CII (sustancias controladas), Aspirin es CIII, mientras que Lipitor y Penicillin no tienen clasificación DEA en el catálogo.

Los cruces muestran que el monto de facturación es prácticamente homogéneo entre categorías: las medias por tipo de admisión varían en menos de 107 USD y por género en 140 USD.

5. Hallazgos y Recomendaciones

Hipótesis validadas

H1: "Las admisiones de emergencia tienen mayor monto de facturación que las electivas." → Rechazada. Prueba t: $t = -0,72$; $p = 0,472$. No existe diferencia significativa (Emergency 25.505 USD vs Elective 25.612 USD).

H2: "Los pacientes de 65 años o más tienen mayor monto de facturación." → Rechazada. Prueba t: $t = -1,22$; $p = 0,221$. El grupo ≥65 años (25.431 USD) no difiere del grupo <65 años.

H3: "La condición Cáncer se asocia con mayor facturación." → Rechazada e invertida. Prueba t: $t = -2,89$; $p = 0,004$. Cáncer presenta la media más baja (25.152 USD).

H4: "El medicamento prescrito se asocia con diferencias en la facturación." → Rechazada. Las medias de Billing Amount por medicamento son prácticamente idénticas (~25.500 USD), sin diferencias significativas; las correlaciones entre Medication_* y Billing Amount son ~0.

Nuevas variables y segmentos propuestos

- Duración de la estancia (días): derivada de la diferencia entre fecha de alta y fecha de admisión (media 15,5 días).
- Segmento de facturación: bajo (≤ 17.196,95 USD), medio (17.196,95–33.397,60 USD) y alto (≥ 33.397,60 USD), por terciles.
- Segmento de edad: 0–17, 18–34, 35–49, 50–64 y 65+ años.
- Atributos farmacológicos por medicamento (PHARM_CLASSES, DOSAGEFORMNAME, ROUTENAME, DEASCHEDULE) incorporados desde el catálogo Drugs.
- Num_Medicamentos: conteo de medicamentos activos por admisión (constante = 1).

Hallazgos y recomendaciones (formato evidencia–acción)

Hallazgo 1. Facturación independiente de la edad y la estancia.

Evidencia: las correlaciones entre Billing Amount y Age (r = -0,003) y entre Billing Amount y duración de la estancia (r = -0,005) son prácticamente nulas; las medias por segmento de edad varían menos de 1.300 USD.

Acción: la facturación no se explica por la edad ni por la duración de la estancia; se recomienda incorporar variables clínicas y de procedimiento (tipo de tratamiento, medicamento, días en UCI) para modelar el costo.

Hallazgo 2. Cáncer presenta la facturación media más baja.

Evidencia: Cáncer (n = 9.140) tiene media de 25.152 USD frente a 25.610 USD del resto de condiciones; prueba t significativa (p = 0,004).

Acción: revisar la codificación de los procedimientos oncológicos y los esquemas de tratamiento; es posible que los costos asociados a Cáncer se registren bajo otras partidas o que exista un patrón de facturación diferenciado que requiera auditoría administrativa.

Hallazgo 3. Homogeneidad de la facturación entre grupos.

Evidencia: no hay diferencias significativas por tipo de admisión ($p = 0,472$), género ni resultado de prueba; las medias por tipo de admisión varían menos de 107 USD.

Acción: el monto facturado es independiente del tipo de admisión y del resultado de la prueba; se sugiere revisar la estructura tarifaria del hospital, ya que la facturación parece estandarizada y no refleja la complejidad clínica.

Hallazgo 4. Asociación entre facturación y facturas negativas.

Evidencia: Billing Amount se correlaciona con Factura_Negativa ($r = -0,08$), la correlación más alta observada con la facturación; existen 106 facturas con monto negativo.

Acción: auditar los 106 registros con monto negativo, ya que representan anomalías administrativas que distorsionan los totales y deben corregirse en origen.

Hallazgo 5. El medicamento no explica la facturación.

Evidencia: las medias de Billing Amount por medicamento son prácticamente idénticas (~25.500 USD) y las correlaciones entre Medication_* y Billing Amount son ~0.

Acción: la prescripción farmacológica no es un determinante del costo de la admisión; para modelar costos se requieren variables de procedimiento y consumo de recursos.

Hallazgo adicional. Los nulos del dataset enriquecido son MNAR (información no disponible en el catálogo) y no se imputan, preservando la trazabilidad del cruce.

6. Anexos

Anexo A. Código principal utilizado en el análisis

src/main.py

```
from repositorios.cargador import CargadorDataset
from servicios.calidad import Diagnosticador, AnalizadorOutliers, Validador
from servicios.limpieza import MarcadorFacturasNegativas, NormalizadorTexto,
TratadorDuplicados
from utilidades.constantes import (
    COLUMNAS_NUMERICAS,
    COLUMNAS_TEXTO,
    COLUMNA_FLAG_FACTURA_NEGATIVA,
    ARCHIVO_CRUDO,
    ARCHIVO_LIMPIO,
)


class PipelineLimpieza:
    def __init__(self, archivoCrudo, archivoLimpio):
        self.archivoCrudo = archivoCrudo
        self.archivoLimpio = archivoLimpio
        self.cargador = CargadorDataset(archivoCrudo)
        self.diagnosticador = Diagnosticador(COLUMNAS_NUMERICAS,
COLUMNA_FLAG_FACTURA_NEGATIVA)
        self.tratadorDuplicados = TratadorDuplicados()
        self.normalizadorTexto = NormalizadorTexto(COLUMNAS_TEXTO)
        self.marcadorNegativas = MarcadorFacturasNegativas(COLUMNA_FLAG_FACTURA_NEGATIVA)
        self.analizadorOutliers = AnalizadorOutliers(COLUMNAS_NUMERICAS,
COLUMNA_FLAG_FACTURA_NEGATIVA)
        self.validador = Validador(COLUMNA_FLAG_FACTURA_NEGATIVA)

    def ejecutar(self):
        df = self.cargador.cargar()
        self.diagnosticador.diagnosticar(df, "Dataset crudo")
        df = self.tratadorDuplicados.quitar(df)
        df = self.normalizadorTexto.normalizar(df)
        df = self.marcadorNegativas.marcar(df)
        self.analizadorOutliers.revisarIQR(df)
        self.diagnosticador.diagnosticar(df, "Dataset limpio")
        self.validador.validar(df)
        df.to_csv(self.archivoLimpio, index=False, float_format="%.2f")
        print("Archivo guardado:", self.archivoLimpio)


if __name__ == "__main__":
    PipelineLimpieza(ARCHIVO_CRUDO, ARCHIVO_LIMPIO).ejecutar()
```

src/main_cruce.py

```
import pandas as pd

from repositorios.cargador import CargadorDataset
from servicios.limpieza import NormalizadorDrugs
```

```
from servicios.cruce import EnriquecedorHealthcareDrugs
from utilidades.constantes import (
    ARCHIVO_TRANSFORMADO,
    ARCHIVO_ENRIQUECIDO,
    ARCHIVO_ANALISIS,
    ARCHIVO_DRUGS_PRODUCT,
    MEDICAMENTOS_MAPEO,
)


class PipelineCruce:
    def __init__(self):
        self.cargadorHealthcare = CargadorDataset(ARCHIVO_TRANSFORMADO)
        self.cargadorDrugs = CargadorDataset(ARCHIVO_DRUGS_PRODUCT, encoding="latin-1")
        self.normalizadorDrugs = NormalizadorDrugs()
        self.enriquecedor = None

    def ejecutar(self):
        print("=== Cargando healthcare transformado ===")
        dfHealthcare = self.cargadorHealthcare.cargar()
        print("Filas:", len(dfHealthcare), "Columnas:", len(dfHealthcare.columns))

        print("\n=== Cargando Drugs_product.csv ===")
        dfDrugs = self.cargadorDrugs.cargar()
        print("Filas:", len(dfDrugs), "Columnas:", len(dfDrugs.columns))
        print("Duplicados:", dfDrugs.duplicated().sum())
        print("Nulos totales:", int(dfDrugs.isna().sum().sum()))
        print("Nulos por columna:")
        print(dfDrugs.isna().sum().sort_values(ascending=False).head(6).to_string())

        print("\n=== Normalizando Drugs ===")
        dfDrugs = self.normalizadorDrugs.normalizar(dfDrugs)

        print("\n=== Enriqueciendo healthcare con Drugs ===")
        self.enriquecedor = EnriquecedorHealthcareDrugs(dfDrugs)
        print(self.enriquecedor.resumen_cobertura().to_string(index=False))

        dfEnriquecido, perfiles = self.enriquecedor.enriquecer(dfHealthcare)
        print("\nForma enriquecida:", dfEnriquecido.shape)
        print("Columnas nuevas:", [c for c in dfEnriquecido.columns if c not in
dfHealthcare.columns])

        print("\n=== Diagnostico del dataset cruzado ===")
        print("Duplicados:", dfEnriquecido.duplicated().sum())
        print("Nulos totales:", int(dfEnriquecido.isna().sum().sum()))
        nulos = dfEnriquecido.isna().sum()
        print("Columnas con nulos:\n", nulos[nulos > 0].to_string())
        print("\nDescribe numericas:")
        print(dfEnriquecido[["Age", "Billing Amount",
"Num_Medicamentos"]].describe().round(2).to_string())

        print("\nClasificacion de faltantes: los NaN en columnas enriquecidas son MNAR")
        print("(informacion no disponible en el catalogo, no error de captura). No se
imputan.")

        print("\n=== Outliers (IQR) del cruzado ===")
        from servicios.calidad import AnalizadorOutliers
        AnalizadorOutliers(["Age", "Billing Amount"]).revisarIQR(dfEnriquecido)

        dfEnriquecido.to_csv(ARCHIVO_ENRIQUECIDO, index=False)
        print("Archivo guardado:", ARCHIVO_ENRIQUECIDO)
```

```
        print("\n=== Analisis edad x medicamento ===")
        flags = [f"Medication_{m}" for m in MEDICAMENTOS_MAPEO if f"Medication_{m}" in
dfEnriquecido.columns]
        filas = []
        for segmento in sorted(dfEnriquecido["Age Segmento"].unique()):
            sub = dfEnriquecido[dfEnriquecido["Age Segmento"] == segmento]
            fila = {"Age Segmento": int(segmento), "Admisiones": len(sub)}
            for f in flags:
                fila[f] = int((sub[f] == 1).sum())
            filas.append(fila)
        dfAnalisis = pd.DataFrame(filas)
        dfAnalisis.to_csv(ARCHIVO_ANALISIS, index=False)
        print(dfAnalisis.to_string(index=False))
        print("Archivo guardado:", ARCHIVO_ANALISIS)

        # resumen de enfermedades por medicamento activo
        print("\n=== Medicamento vs Medical Condition Nivel (conteos) ===")
        for m in MEDICAMENTOS_MAPEO:
            col = f"Medication_{m}"
            if col in dfEnriquecido.columns:
                tab = dfEnriquecido[dfEnriquecido[col] == 1]["Medical Condition
Nivel"].value_counts()
                print(f"\n{m}:")
                print(tab.to_string())


if __name__ == "__main__":
    PipelineCruce().ejecutar()
```

src/servicios/limpieza.py

```
class TratadorDuplicados:
    def __init__(self):
        pass

    def quitar(self, df):
        antes = len(df)
        df = df.drop_duplicates().reset_index(drop=True)
        print("Duplicados eliminados:", antes - len(df))
        return df


class NormalizadorTexto:
    def __init__(self, columnasTexto):
        self.columnasTexto = columnasTexto

    def normalizar(self, df):
        for columna in self.columnasTexto:
            df[columna] = df[columna].str.strip().str.title()
        return df


class NormalizadorDrugs:
    """Normaliza columnas de texto del catálogo de fármacos para facilitar el mapeo."""

    COLUMNAS_TEXTO = [
```

```
        "PROPRIETARYNAME",
        "NONPROPRIETARYNAME",
        "SUBSTANCENAME",
        "PHARM_CLASSES",
        "DOSAGEFORMNAME",
        "ROUTENAME",
        "MARKETINGCATEGORYNAME",
        "PRODUCTTYPENAME",
    ]

    def normalizar(self, df):
        for columna in self.COLUMNAS_TEXTO:
            if columna in df.columns:
                df[columna] = df[columna].astype("string").str.strip()
        return df

class MarcadorFacturasNegativas:
    def __init__(self, columnaFlag):
        self.columnaFlag = columnaFlag

    def marcar(self, df):
        negativos = df["Billing Amount"] < 0
        df[self.columnaFlag] = negativos
        print("Facturas negativas marcadas:", int(negativos.sum()))
        print("El monto original se conserva, no se imputa ningun valor.")
        return df
```

src/servicios/calidad.py

```
class Diagnosticador:
    def __init__(self, columnasNumericas, columnaFlag=None):
        self.columnasNumericas = columnasNumericas
        self.columnaFlag = columnaFlag

    def diagnosticar(self, df, etiqueta):
        print(f"=== {etiqueta} ===")
        print("Filas:", len(df))
        print("Duplicados:", df.duplicated().sum())
        print("Nulos:", df.isna().sum().sum())
        if self.columnaFlag is not None and self.columnaFlag in df.columns:
            print("Facturas negativas marcadas:", int(df[self.columnaFlag].sum()))
            df = df.loc[~df[self.columnaFlag]]
        print(df[self.columnasNumericas].describe().round(2))


class AnalizadorOutliers:
    def __init__(self, columnasNumericas, columnaFlag=None):
        self.columnasNumericas = columnasNumericas
        self.columnaFlag = columnaFlag

    def revisarIQR(self, df):
        base = df
        if self.columnaFlag is not None and self.columnaFlag in df.columns:
            base = df.loc[~df[self.columnaFlag]]
        for columna in self.columnasNumericas:
            q1, q3 = base[columna].quantile([0.25, 0.75])
            iqr = q3 - q1
            limiteInferior = q1 - 1.5 * iqr
```

```
            limiteSuperior = q3 + 1.5 * iqr
            fuera = int(((base[columna] < limiteInferior) | (base[columna] >
limiteSuperior)).sum())
            print(f"{columna}: limites [{limiteInferior:.2f}, {limiteSuperior:.2f}], fuera de
rango: {fuera}")


class Validador:
    def __init__(self, columnaFlag=None):
        self.columnaFlag = columnaFlag

    def validar(self, df):
        print("Duplicados:", df.duplicated().sum())
        print("Nulos:", df.isna().sum().sum())
        negativos = int((df["Billing Amount"] < 0).sum())
        print("Facturas negativas:", negativos)
        if self.columnaFlag is not None and self.columnaFlag in df.columns:
            marcadas = int(df[self.columnaFlag].sum())
            print("Todas marcadas:", marcadas == negativos)
            print("Monto original conservado:", int((df.loc[df[self.columnaFlag], "Billing
Amount"] < 0).sum()) == negativos)
        print("Edades fuera de rango:", int((~df["Age"].between(0, 120)).sum()))
```

src/servicios/cruce.py

```
import pandas as pd

from utilidades.constantes import MEDICAMENTOS_MAPEO, COLUMNAS_DRUGS, DRUGS_NOMINALES_ONEHOT


def _coincide(row, terminos):
    partes = []
    for col in ["NONPROPRIETARYNAME", "PROPRIETARYNAME", "SUBSTANCENAME"]:
        v = row.get(col, "")
        if pd.isna(v):
            continue
        partes.append(str(v))
    texto = " ".join(partes).lower()
    return any(t in texto for t in terminos)


class EnriquecedorHealthcareDrugs:
    def __init__(self, dfDrugs):
        self.dfDrugs = dfDrugs.copy()
        # columna auxiliar de texto combinado en minúsculas para matching vectorizado
        cols = ["NONPROPRIETARYNAME", "PROPRIETARYNAME", "SUBSTANCENAME"]
        self.dfDrugs["_texto"] = ""
        for c in cols:
            if c in self.dfDrugs.columns:
                self.dfDrugs["_texto"] = self.dfDrugs["_texto"] + " " +
self.dfDrugs[c].fillna("").astype(str)
        self.dfDrugs["_texto"] = self.dfDrugs["_texto"].str.lower()

    def _mask(self, terminos):
        m = pd.Series(False, index=self.dfDrugs.index)
        for t in terminos:
            m = m | self.dfDrugs["_texto"].str.contains(t, regex=False, na=False)
        return m
```

```
    def construir_perfil(self, terminos):
        """Devuelve un dict con valores representativos (moda) para un medicamento."""
        mask = self._mask(terminos)
        sub = self.dfDrugs.loc[mask]
        perfil = {"n_productos": len(sub)}
        for col in COLUMNAS_DRUGS:
            if col in sub.columns and len(sub) > 0:
                serie = sub[col].dropna()
                if len(serie) > 0:
                    perfil[col] = serie.mode().iloc[0]
                else:
                    perfil[col] = None
            else:
                perfil[col] = None
        # clases terapéuticas únicas (top)
        if "PHARM_CLASSES" in sub.columns and len(sub) > 0:
            clases = set()
            for v in sub["PHARM_CLASSES"].dropna():
                for c in str(v).split(","):
                    c = c.strip()
                    if c:
                        clases.add(c)
            perfil["CLASES_UNICAS"] = "; ".join(sorted(clases)[:8])
        else:
            perfil["CLASES_UNICAS"] = None
        return perfil

    def enriquecer(self, dfHealthcare):
        df = dfHealthcare.copy()
        perfiles = {}
        for medicamento, terminos in MEDICAMENTOS_MAPEO.items():
            perfiles[medicamento] = self.construir_perfil(terminos)

        # columnas enriquecidas por medicamento (solo donde el flag es 1)
        for medicamento in MEDICAMENTOS_MAPEO:
            colFlag = f"Medication_{medicamento}"
            perfil = perfiles[medicamento]
            for attr in ["PHARM_CLASSES", "DOSAGEFORMNAME", "ROUTENAME", "DEASCHEDULE",
"CLASES_UNICAS"]:
                colNueva = f"{colFlag}_{attr}"
                df[colNueva] = None
                if colFlag in df.columns:
                    mask = df[colFlag] == 1
                    df.loc[mask, colNueva] = perfil.get(attr)

        # columna combinada: clases terapéuticas de los medicamentos activos
        def clases_activas(fila):
            clases = []
            for medicamento in MEDICAMENTOS_MAPEO:
                colFlag = f"Medication_{medicamento}"
                if colFlag in fila and fila[colFlag] == 1:
                    p = perfiles[medicamento]
                    if p.get("CLASES_UNICAS"):
                        clases.append(f"{medicamento}: {p['CLASES_UNICAS']}")
            return " | ".join(clases) if clases else None

        df["Medicamentos_Clases_Terapeuticas"] = df.apply(clases_activas, axis=1)

        # conteo de medicamentos activos por fila
        flags = [f"Medication_{m}" for m in MEDICAMENTOS_MAPEO if f"Medication_{m}" in
```

```
df.columns]
        df["Num_Medicamentos"] = df[flags].sum(axis=1) if flags else 0

        return df, perfiles

    def resumen_cobertura(self):
        out = []
        for medicamento, terminos in MEDICAMENTOS_MAPEO.items():
            p = self.construir_perfil(terminos)
            out.append({"Medicamento": medicamento, "Productos_en_catalogo":
p["n_productos"]})
        return pd.DataFrame(out)
```

src/repositorios/cargador.py

```
import pandas as pd


class CargadorDataset:
    def __init__(self, archivo, encoding=None):
        self.archivo = archivo
        self.encoding = encoding

    def cargar(self):
        if self.encoding:
            return pd.read_csv(self.archivo, encoding=self.encoding)
        return pd.read_csv(self.archivo)
```

src/utilidades/constantes.py

```
from pathlib import Path

CARPETA_UTILIDADES = Path(__file__).resolve().parent
CARPETA_CODIGO = CARPETA_UTILIDADES.parent
CARPETA_PROYECTO = CARPETA_CODIGO.parent
CARPETA_DATA = CARPETA_PROYECTO / "data"
CARPETA_DATA.mkdir(parents=True, exist_ok=True)

ARCHIVO_CRUDO = CARPETA_DATA / "healthcare_dataset_original.csv"
ARCHIVO_LIMPIO = CARPETA_DATA / "healthcare_dataset_limpio.csv"
ARCHIVO_TRANSFORMADO = CARPETA_DATA / "healthcare_dataset_transformado.csv"
ARCHIVO_ENRIQUECIDO = CARPETA_DATA / "healthcare_dataset_enriquecido.csv"
ARCHIVO_ANALISIS = CARPETA_DATA / "analisis_edad_medicamento.csv"

# Dataset de farmacos (catalogo) usado para el cruce
ARCHIVO_DRUGS_PRODUCT = Path(
    r"C:\Users\major\OneDrive\Documentos\manuel\Universidad\octavo semestre\mineria de
datos\Proyecto healtcare\proyecto\data\raw\Drugs_product.csv"
)
ARCHIVO_DRUGS_LIMPIO = CARPETA_DATA / "drugs_product_limpio.csv"
ARCHIVO_DRUGS_TRANSFORMADO = CARPETA_DATA / "drugs_product_transformado.csv"

COLUMNAS_NUMERICAS = ["Age", "Billing Amount"]
COLUMNAS_TEXTO = ["Name", "Doctor", "Hospital"]
COLUMNA_FLAG_FACTURA_NEGATIVA = "Factura_Negativa"

# Mapeo: flag one-hot del healthcare -> terminos de busqueda en Drugs_product
```

```
MEDICAMENTOS_MAPEO = {
    "Aspirin": ["aspirin", "acetylsalicylic"],
    "Ibuprofen": ["ibuprofen"],
    "Lipitor": ["lipitor", "atorvastatin"],
    "Paracetamol": ["paracetamol", "acetaminophen"],
    "Penicillin": ["penicillin"],
}

COLUMNAS_DRUGS = [
    "SUBSTANCENAME",
    "NONPROPRIETARYNAME",
    "PHARM_CLASSES",
    "DOSAGEFORMNAME",
    "ROUTENAME",
    "DEASCHEDULE",
    "MARKETINGCATEGORYNAME",
    "PRODUCTTYPENAME",
]

DRUGS_NOMINALES_ONEHOT = [
    "PRODUCTTYPENAME",
    "MARKETINGCATEGORYNAME",
]
```

src/analisis_correlaciones.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns


def analizar_correlaciones(
    df,
    columnas=None,
    metodo='pearson',
    top_n=15,
    figsize=(9, 7),
    annot=False,
    fmt='.2f',
    triangulo=True,
    cmap='coolwarm',
    centro=0,
    guardar_ruta=None,
    mostrar=True,
):
    if columnas is None:
        df_num = df.select_dtypes(include=['number']).copy()
    else:
        df_num = df[columnas].select_dtypes(include=['number']).copy()

    matriz_corr = df_num.corr(method=metodo)

    pares = []
    cols = matriz_corr.columns
    for i in range(len(cols)):
        for j in range(i + 1, len(cols)):
            r = matriz_corr.iloc[i, j]
```

```
            if pd.notna(r):
                pares.append((cols[i], cols[j], float(r), abs(float(r))))

    pares_ordenados = sorted(pares, key=lambda x: x[3], reverse=True)

    plt.figure(figsize=figsize)
    if triangulo:
        mask = np.triu(np.ones_like(matriz_corr, dtype=bool))
    else:
        mask = None
    sns.heatmap(
        matriz_corr,
        mask=mask,
        annot=annot,
        fmt=fmt,
        cmap=cmap,
        center=centro,
        square=True,
        linewidths=0.5,
        cbar_kws={'shrink': 0.8},
    )
    plt.title(f'Mapa de calor de correlaciones ({metodo})')
    plt.tight_layout()

    if guardar_ruta is not None:
        plt.savefig(guardar_ruta, dpi=150, bbox_inches='tight')
        print(f'Figura guardada: {guardar_ruta}')

    if mostrar:
        try:
            plt.show()
        except Exception:
            pass
    else:
        plt.close()

    top = pares_ordenados[:top_n] if top_n > 0 else pares_ordenados
    df_pares = pd.DataFrame(
        top,
        columns=['Variable 1', 'Variable 2', 'r', '|r|'],
    ).round(4)

    return {
        'matriz': matriz_corr,
        'pares_ordenados': pares_ordenados,
        'top_pares': df_pares,
    }

if __name__ == "__main__":
    import sys
    from pathlib import Path
    sys.path.insert(0, str(Path(__file__).resolve().parent))
    import pandas as pd
    base = Path(__file__).resolve().parent.parent
    df = pd.read_csv(base / "data" / "healthcare_dataset_enriquecido.csv")
    cols = [
        "Age", "Billing Amount", "Medical Condition Nivel", "Admission Type Nivel",
        "Factura_Negativa",
        "Medication_Aspirin", "Medication_Ibuprofen", "Medication_Lipitor",
        "Medication_Paracetamol", "Medication_Penicillin",
    ]
```

```
cols = [c for c in cols if c in df.columns]
res = analizar_correlaciones(
    df, columnas=cols, top_n=15, annot=True, figsize=(9, 7),
    triangulo=False, mostrar=False,
    guardar_ruta=str(base / "docs" / "heatmap_correlaciones.png"),
)
print(res["top_pares"].to_string(index=False))
```

Referencias

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS.

Han, J., Kamber, M., & Pei, J. (2012). Data mining: Concepts and techniques (3.ª ed.). Elsevier.

prasad22. (2024). Healthcare dataset [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/prasad22/healthcare-dataset

U.S. Food and Drug Administration. (2024). Drugs@FDA (Drugs_product) [Conjunto de datos]. https://www.fda.gov/drugs/drug-approvals-and-databases/drugsfda-data-files